

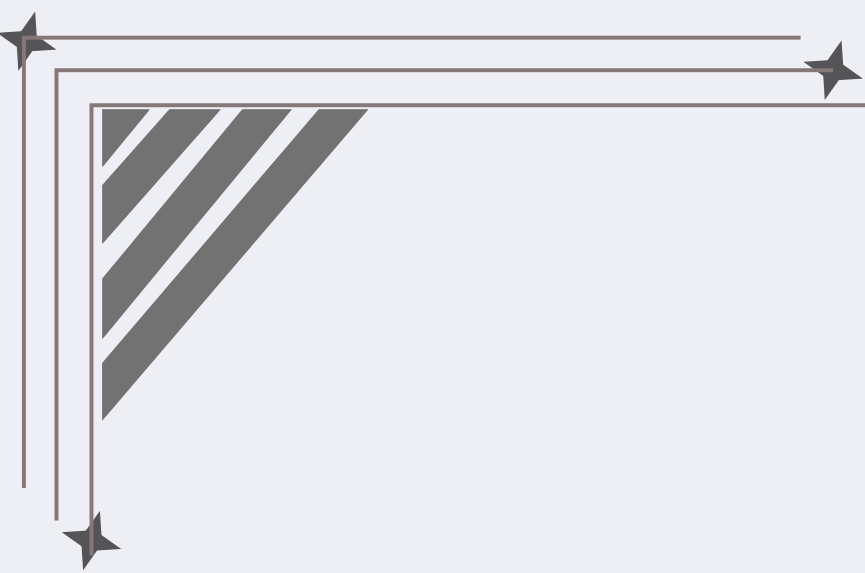


# SUBTITLE GENERATOR



DHRITI REDDY - SE23UARI031  
HIMASREE DINTAKURTHY - SE23UARI032  
DIVYA SRI N - SE23UARI033  
SAIKRUTHI REDDY - SE23UARI037  
HARSHITHA KOLLA - SE23UARI065

*NLP | CS3126*



# MOTIVATION



Manual subtitle creation is labor-intensive and prone to errors. Subtitles ensure accessibility for the hearing impaired, save time for content creators, and enhance video comprehension. Automating this process optimizes efficiency and provides multimedia content access to a broader audience, eliminating the manual subtitle creation burden.

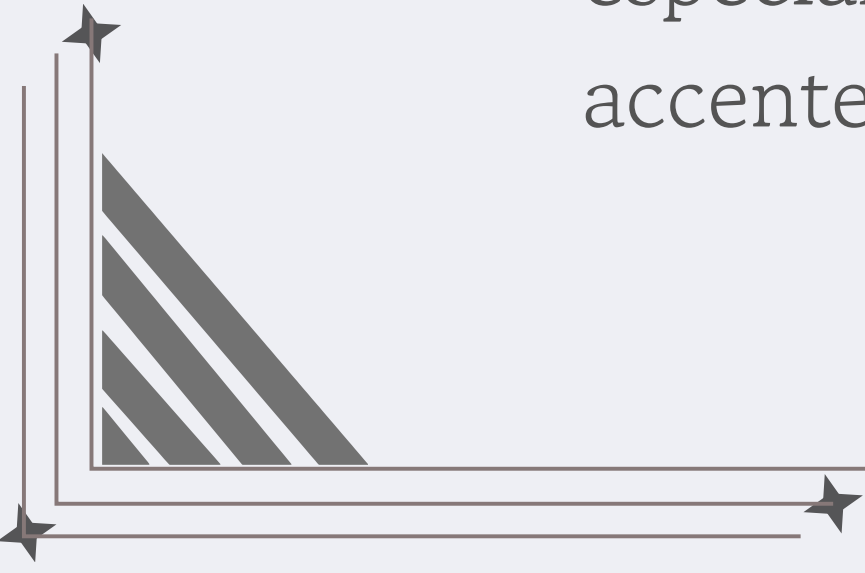




# PROBLEM STATEMENT



Manual Subtitle Creation is Inefficient:

- Labor-intensive and time-consuming process.
  - Prone to human error, especially with fast-paced or accented speech.
- 

Limitations of Existing Automation:

- Current solutions often fail with low-quality or noisy audio.
  - Lack of robust support for regional languages and accurate synchronization.
- 
- 

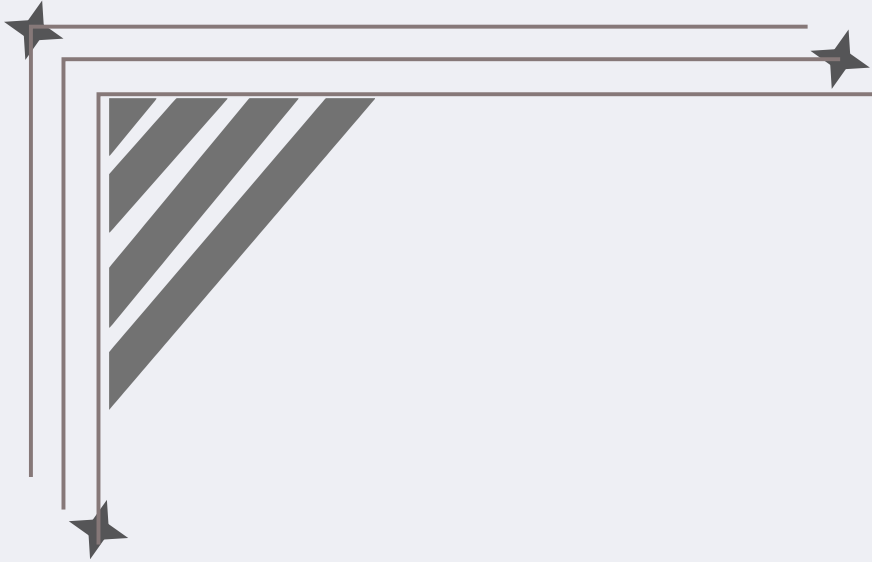
# RELATED WORK

## Existing Tools & Limitations

- YouTube Auto-caption → Limited language support
- Commercial tools (Rev, Otter.ai) → Accurate but expensive
- Open-source tools → Free but poor accuracy

## Research Foundation

- Whisper Model (OpenAI) - Our choice: 680K hours trained, 99+ languages
- Wav2Vec2 (Meta AI) - Alternative: Self-supervised learning approach
- Traditional ASR → Complex pipeline vs End-to-End → Simplified



# WHY WHISPER

- 
- 
- Best multilingual accuracy
  - Free & open-source
  - Robust to noise and accents
  - Directly compatible with our pipeline



# INPUT

The system is designed to be highly flexible in its input sources.

Supported Audio Formats:

mp3, wav, m4a, aac, ogg, flac, wma

Supported Video Formats (Audio is automatically extracted):

mp4, mkv, avi, mov, flv, wmv, 3gp, mpeg, webm

## **Input Processing:**

The input file is automatically converted to a standardized, mono, 16kHz WAV file using FFmpeg for consistent processing.





# DATASETS

Real-World Testing Performed



English: TED Talks (clean studio audio)



Hindi: Suhani Shah show(casual speech with accents)



French: Noisy field recordings (challenging conditions)



# APPROACH

Our automated pipeline consists of four key stages:

1. **Audio Preprocessing:** Enhances audio quality for accurate transcription.
2. **Speech-to-Text (STT) Transcription:** Converts speech to text with timestamps.
3. **Text Post-Processing:** Cleans and refines the transcribed text.
4. **Subtitle File Generation:** Compiles everything into a final .srt file.

# APPROACH

Input  
Audio/Video File



Audio  
Preprocessing



STT Model



Text Post-  
Processing

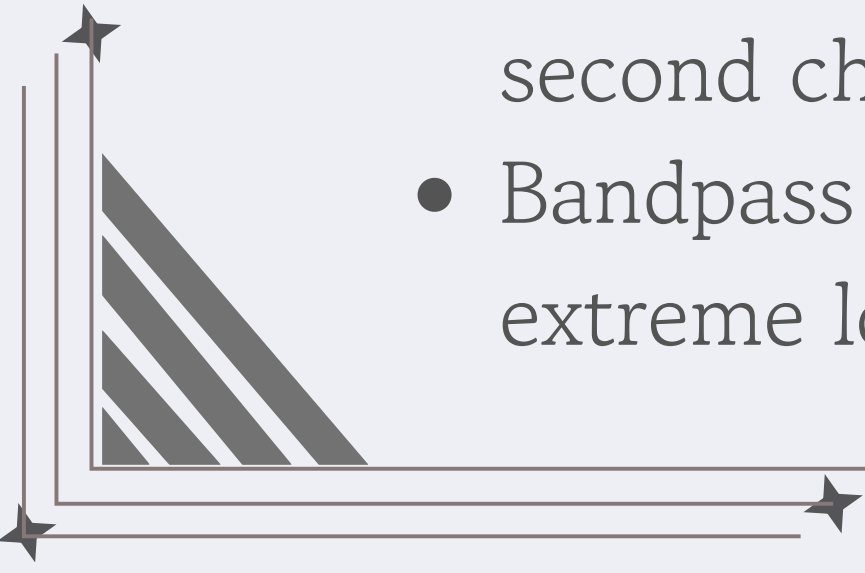


SRT Subtitle File  
Output





# TECHNICAL DEEP DIVE

- 
- 
- Format Conversion & Standardization: Using FFmpeg to convert any input to a 16kHz, mono WAV file.
  - Noise Reduction: Employing the noisereduce library with a high prop\_decrease (0.9-1.0) to aggressively remove background noise.
  - Voice Activity Detection (VAD): Using librosa.effects.split to detect and isolate only the segments containing speech, removing silent parts.
  - Audio Chunking: Splitting the cleaned audio into manageable 30-second chunks to feed into the STT model efficiently.
  - Bandpass Filtering: Applying a safe filter (70 Hz - 8000 Hz) to remove extreme low and high frequencies outside the human speech range.
- 
- 



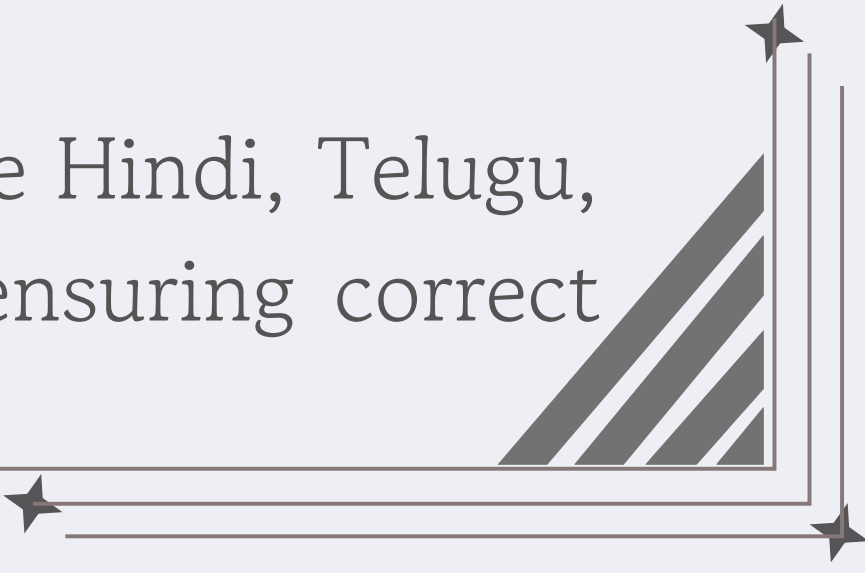
# TECHNICAL DEEP DIVE



## Speech-to-Text Model:

- Model: faster-whisper (An optimized implementation of OpenAI's Whisper).
  - Variant: large-v3 for best-in-class multilingual accuracy.
  - Features: Automatically detects language, provides timestamps for each segment, and runs on GPU (cuda) for speed.
- 

## Text Post-Processing:

- Cleaning: Removal of excessive punctuation, filler words (uh, um), and extra whitespace.
  - Indic Script Normalization: Specialized rules for languages like Hindi, Telugu, and Tamil to fix spacing issues around virama characters, ensuring correct script rendering.
- 
- 



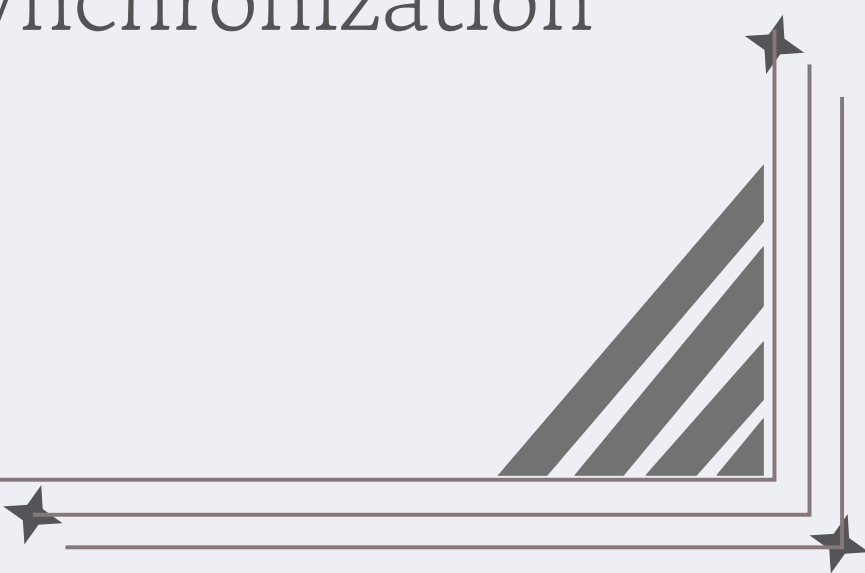
# EXPERIMENTS



## Testing Methodology

- **Multiple Languages:** Hindi, French, English, Telugu
  - **Audio Conditions:** Clean → Noisy → Very Noisy
  - **Real Content:** TED Talks, TV shows, field recordings
  - **Parameter Tuning:** Noise reduction, chunk sizes, model selection
- 

## Evaluation Metrics

- 
- Transcription Accuracy
  - Processing Speed
  - Format Compatibility
  - Timestamp Synchronization
- 

# OUTPUT

- ▶ Accuracy: Leveraging the powerful Whisper model, we achieve high transcription accuracy (~90% as targeted).
- ▶ Synchronization: Each line of text is accurately mapped to its corresponding start and end time in the video.
- ▶ Readability: The text is cleaned and formatted for easy reading.
- ▶ Accessibility: Ready to be used with any video player or platform like YouTube.



# RESULT



English

Hindi

French



# ANALYSIS & DISCUSSION

- High accuracy with Whisper Large-V3
- Effective noise handling
- True multilingual support
- Cost-effective solution

## Strengths

- GPU memory constraints for long files
- Accuracy drops with heavy background noise
- Regional accent challenges remain

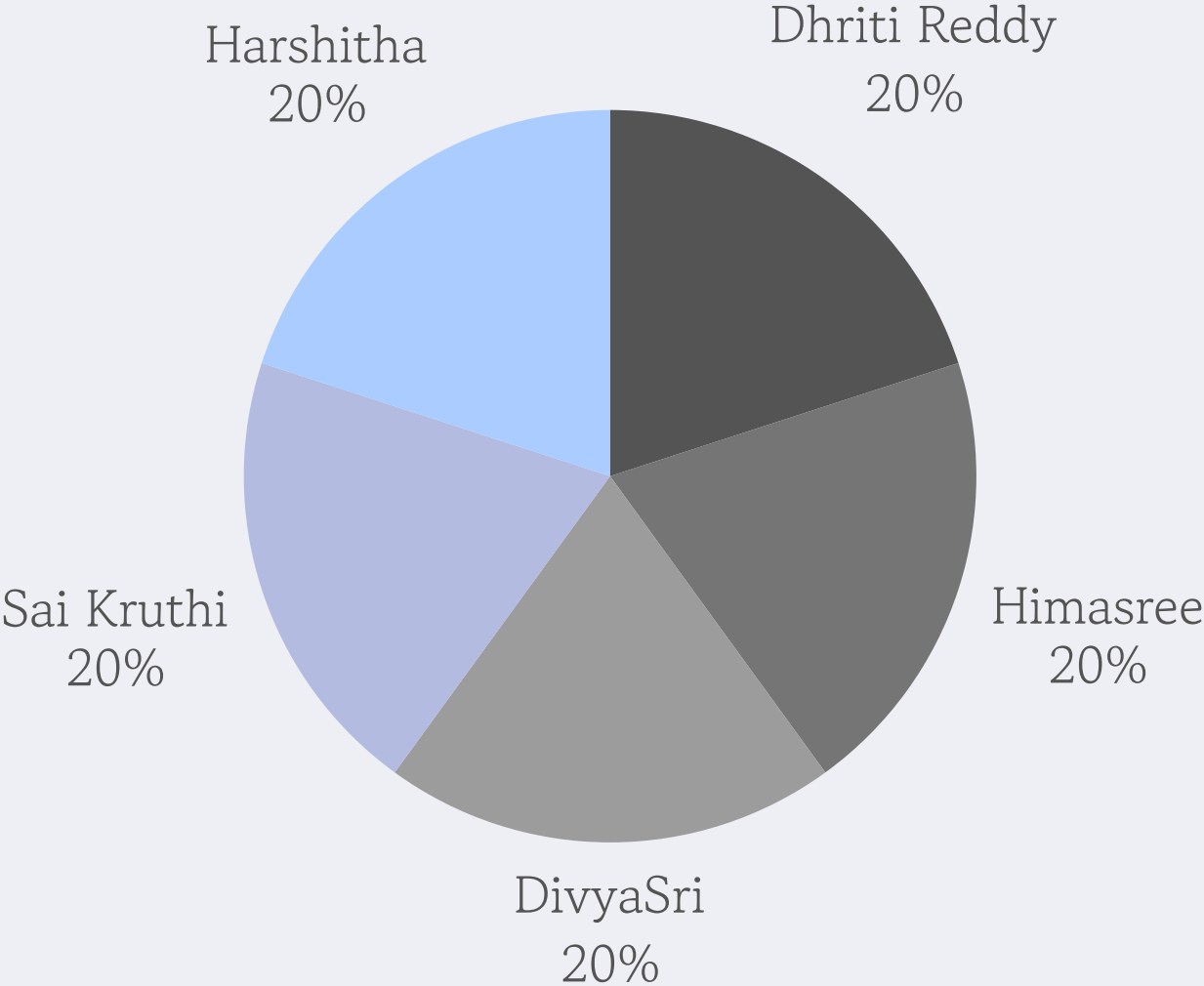
## Limitations

- Whisper provides excellent foundation, but preprocessing is crucial for real-world audio

## Key Insights

# TEAM CONTRIBUTIONS

- Project Setup & Input/Output Handler.
- Text Processing & SRT Builder.
- Audio Preprocessing Engineer.
- Chunking & STT Integration Engineer.
- Pipeline Integration & Quality Assurance.



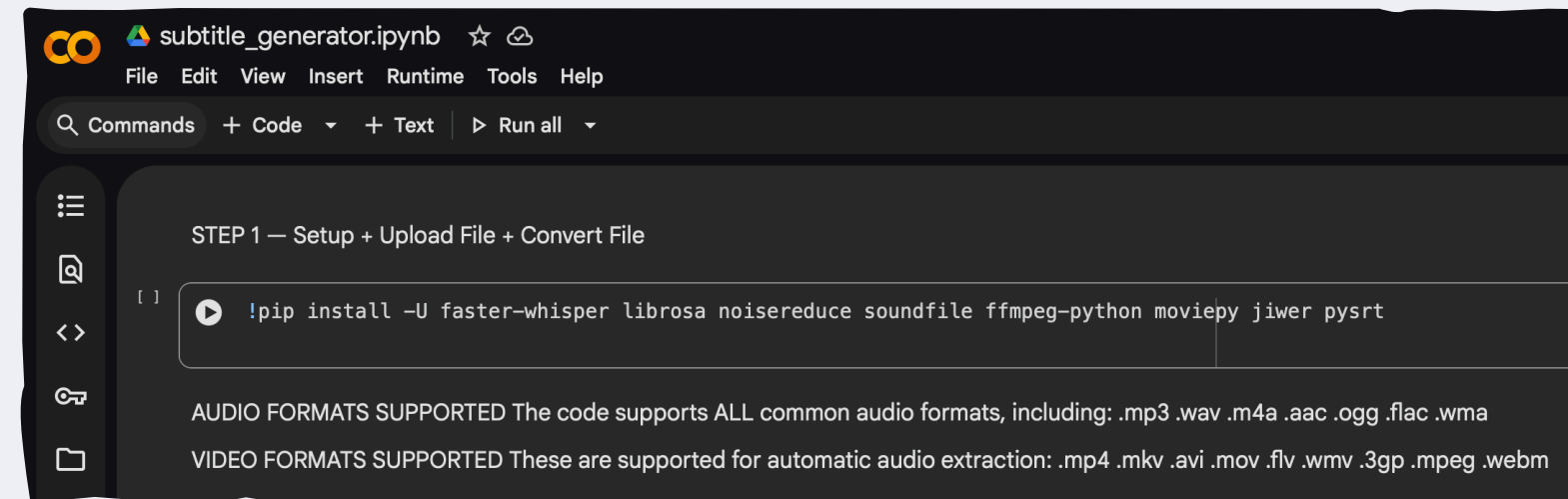
# TEAM CONTRIBUTIONS

|   |                                                                                                                                                                                                                |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | SET UP THE COLAB ENVIRONMENT, MANAGED LIBRARY INSTALLATIONS, IMPLEMENTED THE UNIVERSAL FILE UPLOAD AND CONVERSION SYSTEM USING FFMPEG, AND HANDLED THE FINAL SRT FILE DOWNLOAD.                                |
| 2 | IMPLEMENTED THE CORE AUDIO CLEANING PIPELINE, INCLUDING NOISE REDUCTION (NOISEREDUCE), SILENCE REMOVAL/VAD (LIBROSA), AUDIO NORMALIZATION, AND BANDPASS FILTERING.                                             |
| 3 | DEVELOPED THE LOGIC TO SPLIT THE CLEANED AUDIO INTO FIXED-LENGTH CHUNKS, INTEGRATED THE FASTER-WHISPER LARGE-V3 MODEL FOR TRANSCRIPTION, AND MANAGED THE PROCESSING OF ALL CHUNKS WITH TIMESTAMP OFFSETS.      |
| 4 | DESIGNED AND IMPLEMENTED THE CLEAN_TEXT AND FIX_INDIC_SCRIPT FUNCTIONS TO REFINE THE RAW TRANSCRIPT. BUILT THE FUNCTION TO CONVERT TRANSCRIBED SEGMENTS WITH TIMESTAMPS INTO THE CORRECT SRT FILE FORMAT.      |
| 5 | WROTE THE CORE ORCHESTRATION SCRIPT TO CONNECT ALL MODULES (PREPROCESSING → STT → POST-PROCESSING → SRT). PERFORMED TESTING WITH VARIOUS AUDIO FILES, DEBUGGED ERRORS, AND VALIDATED THE FINAL OUTPUT QUALITY. |

# TIMELINE

- ▶ Research & Setup (Week 1-2)
- ▶ Preprocessing Development (Week 3-4)
- ▶ STT Integration (Week 5-6)
- ▶ Testing & Refinement (Week 7-8)
- ▶ Documentation & Finalization (Week 9-10)

# LIVE DEMO

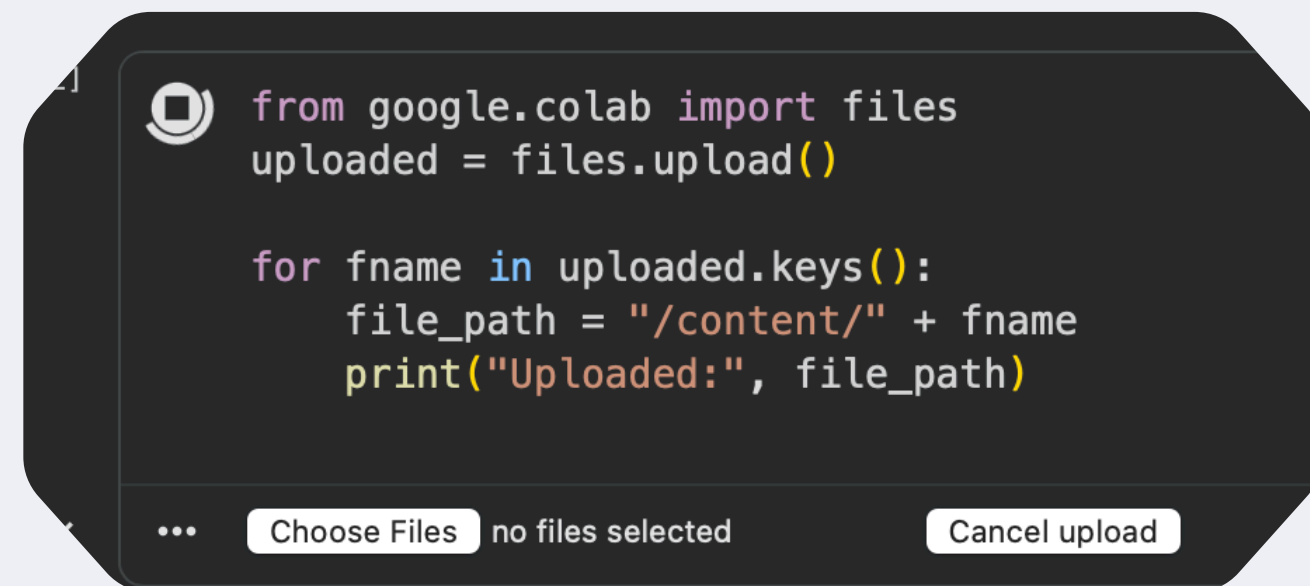


The image shows a Jupyter Notebook interface for a file named 'subtitle\_generator.ipynb'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and tabs for 'Commands', '+ Code', '+ Text', and 'Run all'. The main content area is titled 'STEP 1 — Setup + Upload File + Convert File'. It contains a code cell with the command: `!pip install -U faster-whisper librosa noisereduce soundfile ffmpeg-python moviepy jiwer pysrt`. Below the code cell, there are two text blocks: 'AUDIO FORMATS SUPPORTED The code supports ALL common audio formats, including: .mp3 .wav .m4a .aac .ogg .flac .wma' and 'VIDEO FORMATS SUPPORTED These are supported for automatic audio extraction: .mp4 .mkv .avi .mov .flv .wmv .3gp .mpeg .webm'.

```
!pip install -U faster-whisper librosa noisereduce soundfile ffmpeg-python moviepy jiwer pysrt
```

AUDIO FORMATS SUPPORTED The code supports ALL common audio formats, including: .mp3 .wav .m4a .aac .ogg .flac .wma

VIDEO FORMATS SUPPORTED These are supported for automatic audio extraction: .mp4 .mkv .avi .mov .flv .wmv .3gp .mpeg .webm

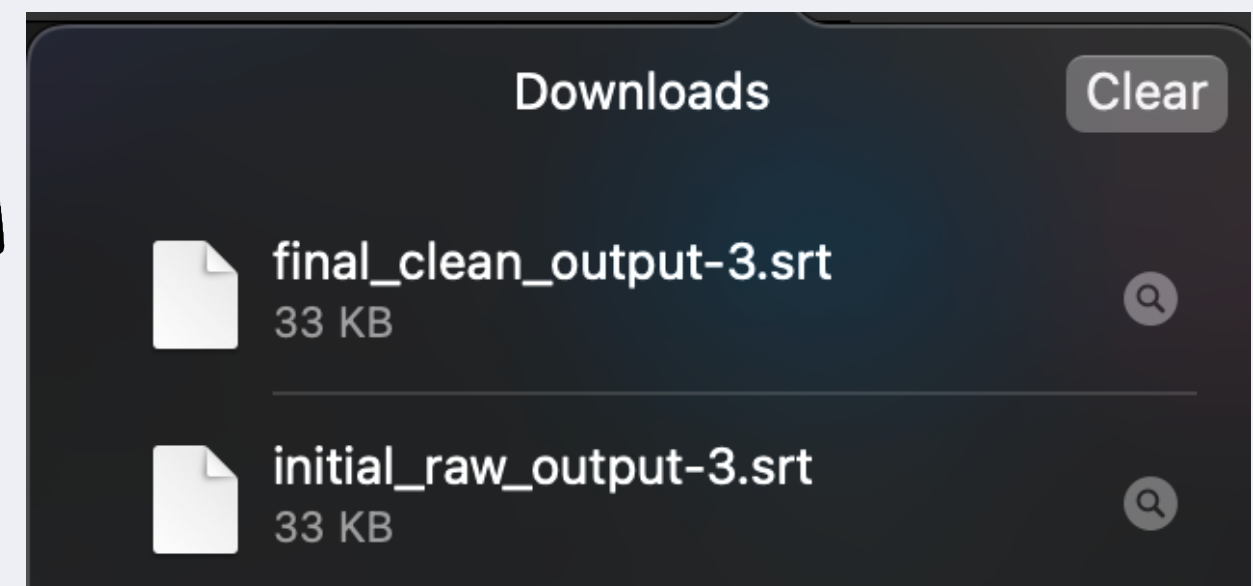


The image shows a Google Colab code cell. The code is as follows: `from google.colab import files`, `uploaded = files.upload()`, `for fname in uploaded.keys():`, `file_path = "/content/" + fname`, and `print("Uploaded:", file_path)`. Below the code, there are three buttons: a three-dot menu button, a 'Choose Files' button, and a 'Cancel upload' button. The text 'no files selected' is displayed between the 'Choose Files' and 'Cancel upload' buttons.

```
from google.colab import files
uploaded = files.upload()

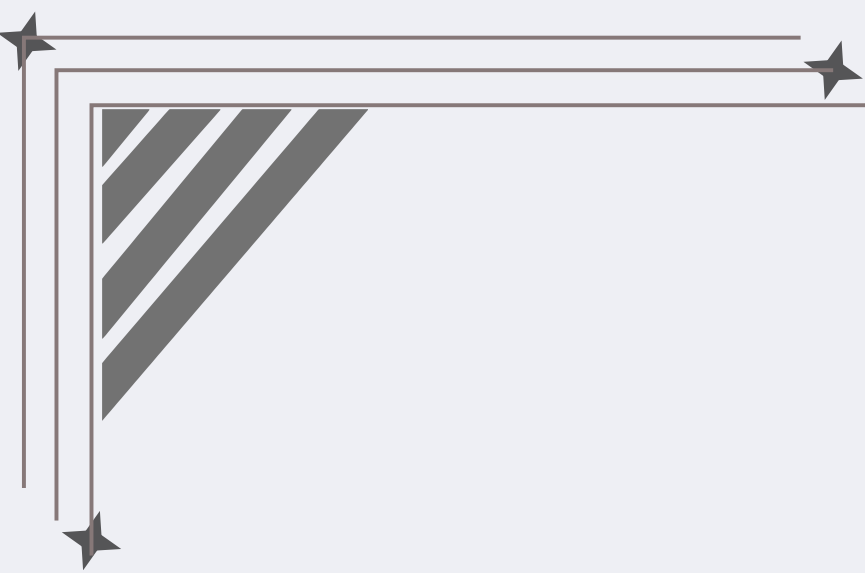
for fname in uploaded.keys():
    file_path = "/content/" + fname
    print("Uploaded:", file_path)
```

Choose Files no files selected Cancel upload



The image shows a 'Downloads' panel with a 'Clear' button in the top right corner. It contains two file entries, each with a document icon, a filename, a size of 33 KB, and a search icon. The first entry is 'final\_clean\_output-3.srt' and the second is 'initial\_raw\_output-3.srt'.

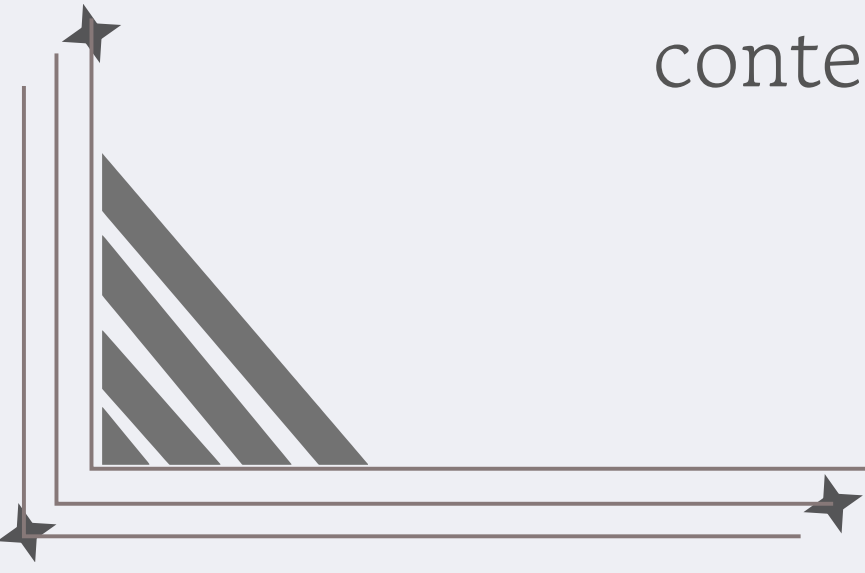
| File Name                | Size  |
|--------------------------|-------|
| final_clean_output-3.srt | 33 KB |
| initial_raw_output-3.srt | 33 KB |



# CONCLUSION



We successfully developed an end-to-end Automatic Subtitle Generator that automates the tedious process of manual subtitle creation. The system efficiently converts audio/video files into accurate, time-synchronized SRT subtitle files, significantly enhancing accessibility and content creator productivity.





THANK YOU

